

PARTIE 1 — SCRIPT DETAILLE DES SLIDES

SLIDE 1

Page de titre

Bonjour, je suis Kenza FOU DALI, stagiaire au sein de CF Consult a Casablanca. Durant ce stage, j'ai eu l'opportunit  de concevoir et developper une application mobile Android pour la gestion de l'inventaire physique de l'entreprise. C'est ce projet que je vais vous presenter aujourd'hui.

SLIDE 2

Presentation de l'organisme d'accueil

CF Consult est un cabinet de conseil et d'expertise comptable base a Casablanca. L'entreprise accompagne ses clients dans la gestion administrative, financiere et organisationnelle. Comme toute structure serieuse, elle doit tenir a jour un inventaire precis de son patrimoine materiel — ordinateurs, mobilier, materiel de bureau — et c'est justement la qu'est nee l'idee de ce projet.

Inventaire Mobile CFC - Application Android

Contexte et besoin

Avant ce projet, le suivi de l'inventaire se faisait manuellement, sur papier ou via des fichiers Excel. Cette methode posait plusieurs limites : risques d'erreurs de saisie, pas de tracabilite en temps reel, et impossibilite d'acceder aux donnees depuis le terrain. L'objectif etait donc de moderniser ce processus en passant a une solution mobile, rapide et fiable. Il ne s'agissait pas d'un probleme bloquant, mais d'une vraie opportunit  de digitalisation.

Kenza FOU DALI, Stagiaire CF Consult Casablanca

Version detaillee : script complet | Questions du jury

Objectifs du projet

Le projet avait trois objectifs principaux. Premierement, centraliser toutes les donnees d'inventaire dans une base de donnees structuree. Deuxiemement, offrir une interface mobile intuitive permettant aux agents de terrain de saisir, modifier et consulter les equipements directement depuis leur telephone. Troisiemement, integrer des fonctionnalites avancees comme le scan de codes-barres et la reconnaissance d'equipements par intelligence artificielle.

SLIDE 5**Planification (Gantt)**

Le projet s'est deroule sur environ deux mois. La premiere semaine etait dediee a l'analyse des besoins et a la redaction du cahier des charges. Ensuite j'ai concu les diagrammes UML — cas d'utilisation, diagramme de classes, sequences. La phase de developpement a occupe la majorite du stage : backend Spring Boot, puis application Android. Les deux dernieres semaines ont servi aux tests et aux corrections. Cette planification m'a permis de livrer un produit fonctionnel dans les delais.

SLIDE 6**Architecture globale (3 tiers)**

L'architecture que j'ai choisie est une architecture trois tiers, qui est le standard pour les applications client-serveur modernes.

Le premier tier est la couche presentation : c'est l'application Android, developpee en Java natif. Elle s'occupe uniquement de l'affichage et des interactions utilisateur.

Le deuxieme tier est la couche metier : c'est le backend Spring Boot, qui tourne sur le serveur. Il contient toute la logique applicative — les regles de gestion, la securite, l'authentification JWT, et l'exposition des API REST.

Le troisieme tier est la couche donnees : une base de donnees PostgreSQL qui stocke de facon persistante tous les equipements, utilisateurs et mouvements.

Cette separation est fondamentale : si demain on veut creer une application web en plus de l'application Android, on reutilise exactement le meme backend sans y toucher.

SLIDE 7**Backend Spring Boot**

Le backend est structure selon les couches classiques de Spring Boot. On a les Controllers REST qui recoivent les requetes HTTP et retournent des reponses JSON. En dessous, les Services qui contiennent la logique metier — par exemple, verifier qu'un equipement n'est pas en doublon avant de l'enregistrer. Ensuite les Repositories qui font l'interface avec la base de donnees via Spring Data JPA. Et enfin les Entites qui sont les classes Java mappees directement sur les tables PostgreSQL grace aux annotations Hibernate comme @Entity, @Table, @Column.

La securite est geree par Spring Security avec JWT — JSON Web Token. Quand l'utilisateur se connecte, le backend genere un token signe qui encode son role et son identite. Ce token est ensuite envoye dans chaque requete dans le header HTTP Authorization: Bearer . Le backend le verifie a chaque appel sans avoir besoin d'interroger la base de donnees, ce qui est tres efficace.

SLIDE 8

Application Android

L'application Android est developpee en Java natif, sans framework tiers comme Flutter ou React Native. J'ai fait ce choix pour avoir un controle total sur les performances et l'acces aux APIs Android.

L'architecture cote Android suit le pattern MVC — Modele Vue Controleur. Les Activities jouent le role de controleurs, les layouts XML sont les vues, et les modeles de donnees representent les entites metier.

Pour communiquer avec le backend, j'utilise Retrofit2, une librairie qui transforme automatiquement les appels API en requetes HTTP et deserialise les reponses JSON en objets Java grace a GSON.

La session utilisateur est geree via SharedPreferences — un stockage cle-valeur local sur le telephone — ou je sauvegarde le token JWT, le role et le nom de l'utilisateur apres connexion.

SLIDE 9

Authentification JWT (flux complet)

Laissez-moi vous expliquer precisement le flux d'authentification. L'utilisateur saisit son login et son mot de passe dans l'application. L'app envoie ces credentials en POST JSON vers l'endpoint /auth/login du backend. Le backend verifie les credentials dans PostgreSQL — le mot de passe est hache en BCrypt pour la securite. Si c'est correct, il genere un JWT signe avec une cle secrete, et le renvoie a l'application. L'application stocke ce token en SharedPreferences. A partir de la, chaque requete suivante inclut ce token dans le header HTTP. Le backend decode et verifie le token a chaque appel grace a un filtre Spring Security qui s'execute avant chaque controleur.

SLIDE 10

Scan de codes-barres (ZXing)

Pour le scan de codes-barres, j'utilise la librairie ZXing — Zebra Crossing — qui est la reference open source pour la lecture de codes-barres et QR codes sur Android. L'integration se fait via IntentIntegrator : on lance une Intent vers l'activite de scan ZXing, elle ouvre la camera, detecte le code, et renvoie le resultat a notre Activity via onActivityResult. Ce numero de serie ou code d'inventaire est ensuite utilise pour rechercher l'equipement correspondant dans la base de donnees via un appel Retrofit vers le backend.

SLIDE 11

IA Gemini (reconnaissance d'equipements)

La fonctionnalite la plus avancee de cette application est la reconnaissance d'equipements par intelligence artificielle. J'ai integre l'API Google Gemini 2.5 Flash — le modele multimodal de Google.

Le flux fonctionne ainsi : l'agent prend une photo d'un equipement avec son telephone, ou en selectionne une depuis la galerie. L'application encode l'image en Base64 et envoie une requete HTTP directement a l'API Gemini — sans passer par notre backend, ce qui simplifie l'architecture. Le prompt que j'envoie a Gemini lui demande d'identifier l'objet et de repondre uniquement en JSON avec les champs : designation, type, marque, etat et description.

Gemini analyse l'image et retourne ces informations. L'application parse le JSON et pre-remplit automatiquement le formulaire d'ajout d'equipement. L'agent n'a plus qu'a verifier et valider. Cette fonctionnalite reduit considerablement le temps de saisie.

SLIDE 12

Fonctionnalites principales

L'application couvre tout le cycle de vie d'un inventaire. Pour les equipements informatiques, on peut creer une fiche avec designation, marque, numero de serie, etat, localisation et responsable. Pour les autres actifs — mobilier, materiel divers — il y a un module separe adapte. On peut consulter la liste, rechercher par mot-cle, modifier une fiche, et la supprimer.

Il y a aussi un module d'export : on peut generer un fichier Excel ou PDF de l'inventaire complet depuis l'application.

La gestion des droits est implementee : un utilisateur simple peut consulter et creer, mais seul un administrateur peut supprimer ou acceder au module d'administration des comptes utilisateurs.

SLIDE 13

Base de donnees PostgreSQL

La base de donnees est PostgreSQL, un SGBD relationnel robuste et open source. Le schema comporte plusieurs tables principales : utilisateurs avec les colonnes login, password hashe, role, nom, prenom ; equipements avec toutes les caracteristiques materielles ; autres_actifs pour le mobilier. Les relations sont modelisees avec des cles etrangeres — par exemple un equipement peut etre lie a un responsable dans la table utilisateurs.

Grace a Spring Data JPA et Hibernate, je n'ecris quasiment pas de SQL a la main. Hibernate genere les requetes automatiquement a partir des annotations sur mes classes Java. Et au demarrage de l'application Spring Boot, avec `spring.jpa.hibernate.ddl-auto=update`, le schema se met a jour automatiquement si je modifie mes entites.

SLIDE 14

Technologies utilisees

Pour resumer la stack technique : cote backend, Java 17 avec Spring Boot 3, Spring Security pour l'authentification, Spring Data JPA / Hibernate pour l'ORM, PostgreSQL comme base de donnees, et Maven pour la gestion des dependances. Cote mobile, Android Java avec Retrofit2 pour les appels API, OkHttp pour les requetes directes vers Gemini, ZXing pour le scan, et Material Design 3 pour l'interface. Pour le developpement j'ai utilise Android Studio, IntelliJ IDEA, et Postman pour tester les endpoints REST.

SLIDE 15

Demonstration / Resultats

Concretement, l'application est entierement fonctionnelle. On peut se connecter, naviguer dans l'inventaire, scanner un code-barres pour retrouver un equipement instantanement, prendre une photo et laisser l'IA l'identifier automatiquement, puis valider et enregistrer. Le tout en quelques secondes, la ou la saisie manuelle prenait plusieurs minutes et comportait des risques d'erreurs.

SLIDE 16

Conclusion et perspectives d'evolution

Ce stage m'a permis de travailler sur une solution full-stack complete, de la conception de la base de donnees jusqu'a l'interface mobile, en passant par la mise en place d'une API REST securisee. J'ai eu l'occasion d'appliquer des technologies modernes comme JWT, l'IA generative avec Gemini,

et le scan de codes-barres.

Plusieurs perspectives d'evolution sont envisageables. Premierement, implementer des notifications push via Firebase Cloud Messaging pour alerter les responsables lors d'ajouts ou de modifications. Deuxiemement, ajouter un mode hors ligne complet avec Room Database — la base de donnees SQLite Android — et une synchronisation differee quand le reseau revient. Troisiemement, migrer vers Kotlin, le langage moderne recommande par Google pour Android, qui offre une syntaxe plus concise et une meilleure gestion des coroutines pour l'asynchrone. Quatriemement, ajouter des tests unitaires avec JUnit pour le backend et Espresso pour l'UI Android.

C'est un projet concret qui a une vraie valeur ajoutee pour l'entreprise. Je suis disponible pour vos questions.

PARTIE 2 — QUESTIONS DU JURY (reponses detaillees)

ARCHITECTURE & DESIGN

Q1 : Pourquoi avoir choisi une architecture 3 tiers plutot qu'une architecture 2 tiers directement connectee a la base de donnees depuis Android ?

Une architecture 2 tiers — ou Android se connecte directement a PostgreSQL — poserait des problemes majeurs de securite. Il faudrait exposer les credentials de la base de donnees dans l'application mobile, qui peut etre desassemblee. Avec une architecture 3 tiers, le backend est le seul a acceder a la base de donnees. L'application mobile ne voit jamais les credentials BD. De plus, la logique metier est centralisee : si je change une regle de validation, je la change une seule fois cote backend, pas dans chaque client.

Q2 : Pourquoi Spring Boot pour le backend ?

Spring Boot offre plusieurs avantages. L'auto-configuration reduit considerablement le boilerplate. Spring Security est tres complet pour la gestion de l'authentification et des autorisations. Spring Data JPA simplifie les interactions avec la base de donnees. Et Spring Boot embarque un serveur Tomcat, donc pas besoin de configurer un serveur externe separement. C'est aussi un standard industriel tres utilise en entreprise.

Q3 : Pourquoi PostgreSQL plutot que MySQL ou SQLite ?

SQLite est un fichier local, pas adapte pour une application client-serveur multi-utilisateurs. Entre PostgreSQL et MySQL, j'ai choisi PostgreSQL pour sa robustesse, sa conformite aux standards SQL, et ses meilleures performances sur les requetes complexes. PostgreSQL gere aussi mieux les transactions concurrentes — important quand plusieurs agents font des modifications en meme temps.

SECURITE & AUTHENTIFICATION

Q4 : Expliquez precisement comment fonctionne JWT.

JWT — JSON Web Token — est un standard ouvert. Le token est compose de trois parties encodees en Base64 et separees par des points : le header qui indique l'algorithme de signature (HS256 ici), le payload qui contient les claims — c'est-a-dire les informations : userId, role, date d'expiration — et la signature calculee avec une cle secrete stockee cote serveur. Quand le backend recoit un token, il recalcule la signature et la compare. Si elles correspondent, le token est valide et non falsifie. L'avantage est que c'est stateless : le serveur ne stocke aucune session, ce qui facilite la montee en charge.

Q5 : Comment les mots de passe sont-ils stockés ?

Les mots de passe ne sont jamais stockés en clair. J'utilise BCrypt, qui est un algorithme de hachage adaptatif. BCrypt génère un salt aléatoire pour chaque mot de passe, ce qui signifie que deux utilisateurs avec le même mot de passe auront des hashes différents. Le facteur de coût de BCrypt peut être augmenté pour ralentir les attaques par brute force. Côté Spring Security, BCryptPasswordEncoder gère ça automatiquement.

Q6 : Que se passe-t-il si le token JWT expire ?

Dans l'implémentation actuelle, si le token expire, les appels API retournent une erreur 401 Unauthorized. L'application redirige alors l'utilisateur vers l'écran de connexion pour se reconnecter et obtenir un nouveau token. Une amélioration future serait d'implémenter un refresh token — un second token à longue durée de vie permettant de renouveler le token d'accès sans redemander les credentials.

GEMINI & IA

Q7 : Comment fonctionne l'intégration de l'IA Gemini techniquement ?

L'image capturée ou sélectionnée est d'abord lue comme flux d'octets via `ContentResolver.openInputStream()`. Ces octets sont encodés en Base64 — une représentation textuelle des données binaires. Je construis ensuite un objet JSON selon le format de l'API Gemini : un tableau `contents` contenant un tableau `parts` avec d'abord le prompt textuel, puis l'image inline avec son type MIME `image/jpeg` et les données Base64. Cette requête est envoyée via OkHttp en POST vers l'endpoint Gemini. La réponse JSON contient les candidates, dont j'extrais le texte de la première partie, que je parse à son tour comme JSON pour récupérer les champs de l'équipement.

Q8 : Pourquoi Gemini 2.5 Flash et pas ChatGPT ou un autre modèle ?

Gemini 2.5 Flash présente plusieurs avantages pour ce cas d'usage. C'est un modèle multimodal natif — conçu dès le départ pour traiter images et texte ensemble, pas une extension. Il est très rapide — le suffixe Flash indique un modèle optimisé pour la latence. Il est disponible avec un quota gratuit suffisant pour une démonstration. Et l'API est simple à intégrer via HTTP standard, sans SDK obligatoire.

Q9 : Quels sont les risques de sécurité liés à la clé API Gemini dans l'app Android ?

C'est une question importante. Une clé API directement dans le code d'une application mobile peut être extraite par décompilation du fichier APK. Pour une application en production, il faudrait faire passer tous les appels Gemini par notre propre backend, qui lui détient la clé en variable d'environnement. Le backend expose un endpoint `/ai/analyze` qui reçoit l'image de l'app Android et appelle Gemini en serveur à serveur. Cela garantit que la clé n'est jamais exposée côté client.

ANDROID & TECHNIQUE

Q10 : Pourquoi Java Android natif et pas Flutter ou React Native ?

Flutter et React Native sont des choix valides, mais Java Android natif offre un accès direct et sans couche d'abstraction à toutes les APIs Android. Pour des fonctionnalités comme la caméra, FileProvider, et les permissions, le code natif est plus fiable et mieux documenté. De plus, Java est le langage que j'ai appris en cours, ce qui m'a permis d'être productive plus rapidement.

Q11 : Comment fonctionne le scan de codes-barres techniquement ?

ZXing utilise les APIs de la caméra Android pour capturer des frames vidéo en continu. Chaque frame est analysée par des algorithmes de traitement d'image pour détecter des patterns caractéristiques des codes-barres — les barres verticales pour un code 1D, les carrés pour un QR code. Une fois le pattern détecté, l'algorithme de decodage Reed-Solomon corrige les éventuelles erreurs de lecture et extrait la chaîne de caractères encodée. Dans mon app, j'utilise IntentIntegrator qui délègue tout ce traitement à l'Activity ZXing et me retourne juste le résultat.

Q12 : Comment gerez-vous les erreurs réseau dans l'application ?

Retrofit et OkHttp gèrent les appels de façon asynchrone via des callbacks. Le callback onFailure est appelé en cas d'absence réseau ou de timeout. Le callback onResponse est appelé quand on reçoit une réponse HTTP — même une erreur 4xx ou 5xx. Je vérifie response.isSuccessful() pour distinguer les succès des erreurs HTTP, et j'affiche des messages Toast appropriés à l'utilisateur selon le cas. En mode demo avec le login admin/admin, l'application fonctionne entièrement hors ligne.

Q13 : Quelle est la différence entre Retrofit et OkHttp dans votre code ?

OkHttp est la couche HTTP bas niveau — il gère les connexions TCP, les timeouts, les intercepteurs. Retrofit est une couche d'abstraction au-dessus d'OkHttp qui transforme des interfaces Java annotées en appels HTTP. Retrofit utilise OkHttp en interne. Dans mon code, j'utilise Retrofit pour tous les appels à mon backend Spring Boot — c'est plus propre et maintenable. J'utilise OkHttp directement pour les appels à l'API Gemini parce que le format de requête multimodal est complexe et que je voulais construire le JSON manuellement sans passer par les annotations Retrofit.

Q14 : Comment structure le diagramme de classes de votre application ?

Le diagramme de classes reflète l'architecture en couches. Côté backend, les entités JPA comme Equipement et Utilisateur ont des attributs correspondant aux colonnes BD et des annotations comme @Entity, @Id, @GeneratedValue. Chaque entité a son Repository qui étend JpaRepository — ce qui fournit automatiquement les méthodes CRUD. Les Services dépendent des repositories et contiennent la logique métier.

Les Controllers dependent des services et exposent les endpoints REST. Cote Android, les classes Activity communiquent avec les classes Model via ApiService — l'interface Retrofit.

Q15 : Quelles ameliorations pourriez-vous apporter a ce projet ?

Plusieurs pistes d'amelioration sont envisageables. Premierement, implementer des notifications push via Firebase Cloud Messaging pour alerter les responsables lors d'ajouts ou de modifications. Deuxiemement, ajouter un mode hors ligne complet avec Room Database — la base de donnees SQLite Android — et une synchronisation differee quand le reseau revient. Troisiemement, migrer vers Kotlin, le langage moderne recommande par Google pour Android, qui offre une syntaxe plus concise et une meilleure gestion des coroutines pour l'asynchrone. Quatriemement, ajouter des tests unitaires avec JUnit pour le backend et Espresso pour l'UI Android.